

Strukturierte & halbstrukturierte Daten laden (CSV, JSON)

Section 2 · Data Ingestion and Loading · Quelle: Kurs 1, Kapitel 10–13

CSV und JSON sind die beiden häufigsten Rohdatenformate beim Einstieg in die Bronze-Schicht. Beide werden über dieselbe Funktion – `read_files()` – eingelesen, unterscheiden sich aber deutlich in ihren Eigenheiten: CSV ist flach und trennzeichenbasiert, bringt aber kein eingebettetes Schema mit; JSON ist von Natur aus verschachtelt und erfordert eigene Techniken, um aus *semi-strukturierten* Daten *strukturierte* Spalten zu machen.

1. CSV-Dateien korrekt einlesen

CSV-Dateien (Comma-Separated Values) speichern Daten zeilenweise als Text, wobei Werte durch ein Trennzeichen – meist Komma, häufig aber auch Semikolon, Tabulator oder Pipe (`|`) – getrennt sind. Ruft man `read_files()` ohne weitere Optionen auf, geht die Funktion von den Standardeinstellungen aus (Komma als Trennzeichen, keine Kopfzeile) – bei abweichend formatierten Dateien liefert das dann falsch aufgeteilte oder falsch benannte Spalten. Deshalb müssen die tatsächlichen Formateigenschaften der Quelle explizit als Optionen übergeben werden:

```
-- Ohne Optionen: Spalten werden nicht korrekt erkannt
SELECT *
FROM read_files(
  "/Volumes/dbacademy_ecommerce/v01/raw/sales-csv",
  format => "csv"
)
LIMIT 5;

-- Mit den passenden Optionen für diese Quelle
SELECT *
FROM read_files(
  "/Volumes/dbacademy_ecommerce/v01/raw/sales-csv",
  format => "csv",
  sep    => "|",      -- Spaltentrennzeichen (Standard: ",")
  header => true      -- erste Zeile enthält die Spaltennamen
)
LIMIT 5;
```

Wichtige weitere Optionen von `read_files()` für CSV sind u. a. `encoding` (Zeichensatz der Datei, z. B. `UTF-8` oder `ISO-8859-1`), `quote` und `escape` (Behandlung von in Werten enthaltenen Trennzeichen) sowie `schema`, um die automatische Schema-Ableitung zu umgehen. Letzteres lohnt sich fast immer im produktiven Einsatz: Ohne explizites Schema muss Databricks zunächst einen repräsentativen Teil aller Dateien lesen, um ein gemeinsames Schema zu bestimmen – das kostet Zeit und kann bei uneinheitlichen Quelldateien zu unerwarteten Typen führen. Ein festes Schema ist außerdem Voraussetzung dafür, dass Schema-Abweichungen zuverlässig in der `_rescued_data`-Spalte landen (siehe vorheriges Kapitel dieses Themenordners):

```
SELECT *
FROM read_files(
  '/Volumes/dbacademy/ops/labuser/csv_demo_files/malformed_example_1_data.csv',
  format => "csv",
  sep    => "|",
  header => true,
  schema => '''
    order_id INT,
    email STRING,
    transactions_timestamp BIGINT''' ,
  rescuedDataColumn => "_rescued_data"
);
```

2. JSON: von der verschachtelten Struktur zur Spalte

JSON-Daten bestehen aus Objekten in geschweiften Klammern mit Schlüssel-Wert-Paaren; Werte können wiederum Zeichenketten, Zahlen, boolesche Werte, verschachtelte Objekte oder Arrays sein. Genau diese Verschachtelung macht JSON flexibel, aber auch komplexer zu verarbeiten als ein flaches CSV. Databricks bietet für JSON-formatierte Spalten drei grundsätzliche Ansätze:

Ansatz	Eigenschaft
STRING	JSON bleibt als reiner Text gespeichert; flexibel, aber langsam bei Abfragen, kein erzwungenes Schema.
STRUCT	JSON wird anhand eines definierten Schemas in typisierte, verschachtelte Spalten geparkt; erzwingt Konsistenz und ist deutlich performanter.
VARIANT	Neuerer, offener Datentyp für semi-strukturierte Daten; kombiniert die Flexibilität von STRING mit einer Performance nahe an STRUCT, ohne starres Schema vorauszusetzen.

2.1 JSON-Werte direkt aus einer STRING-Spalte extrahieren

Mit der Doppelpunkt-Syntax `spalte:feld` lassen sich einzelne Felder direkt aus einer JSON-formatierten Textspalte auslesen – auch verschachtelt, mit Punkt- oder Klammer-Notation. Das ist der einfachste Einstieg, eignet sich aber vor allem für Ad-hoc-Abfragen, nicht für performancekritische, wiederkehrende Verarbeitung:

```
SELECT
  decoded_value,
  decoded_value:device,
  decoded_value:traffic_source,
  decoded_value:geo,      -- enthält selbst wieder einen JSON-String
  decoded_value:items     -- enthält ein verschachteltes Array von JSON-Objekten
FROM kafka_events_bronze_decoded
LIMIT 5;
```

2.2 JSON in eine typisierte STRUCT-Spalte umwandeln

Für wiederkehrende, performancekritische Verarbeitung empfiehlt sich die Umwandlung in eine `STRUCT`-Spalte. Der Weg dahin führt über zwei Funktionen: `schema_of_json()` leitet aus einem Beispiel-String automatisch das passende Schema ab, `from_json()` wendet dieses Schema anschließend auf die gesamte Spalte an und erzeugt eine typisierte `STRUCT`-Spalte mit verschachtelten Feldern und Arrays:

```
-- Schritt 1: Schema aus einem Beispielwert ableiten
SELECT schema_of_json(
  '{"device":"Linux","ecommerce":{"purchase_revenue_in_usd":1075.5},"geo":
{"city":"Houston","state":"TX"}}'
) AS schema;

-- Schritt 2: Schema mit from_json() anwenden
CREATE OR REPLACE TABLE kafka_events_bronze_struct AS
SELECT
  * EXCEPT (decoded_value),
  from_json(
    decoded_value,
    'STRUCT<device: STRING, ecommerce: STRUCT<purchase_revenue_in_usd: DOUBLE,
total_item_quantity: BIGINT>,
      geo: STRUCT<city: STRING, state: STRING>,
      items: ARRAY<STRUCT<item_id: STRING, item_name: STRING, price_in_usd: DOUBLE,
quantity: BIGINT>>,
      traffic_source: STRING, user_id: STRING>'
  ) AS value
FROM kafka_events_bronze_decoded;

-- Zugriff auf Felder, verschachtelte Felder und Arrays per Punktnotation
SELECT
  value.device                AS device,
  value.geo.city              AS city,
  value.items,
  array_size(value.items)     AS anzahl_items
FROM kafka_events_bronze_struct;
```

Enthält eine Zeile ein Array mit mehreren Elementen (z. B. mehrere gekaufte Artikel pro Bestellung), lässt sich dieses Array mit `explode()` in einzelne Zeilen auflösen – eine Zeile pro Array-Element. Ist der Array-Wert `NULL`, erzeugt `explode()` dafür keine Zeile; soll trotzdem eine Zeile mit `NULL`-Werten erhalten bleiben, verwendet man stattdessen `explode_outer()`.

```
SELECT
  decoded_key,
  array_size(value.items) AS anzahl_items,
  explode(value.items)   AS einzelnes_item
FROM kafka_events_bronze_struct;
```

2.3 Sonderfall: Base64-kodierte Felder (z. B. Kafka-Events)

Bei Ereignisdaten aus Message-Bussen wie Kafka sind Schlüssel- und Wertfelder häufig Base64-kodiert, um Sonderzeichen und Formatierung beim Transport nicht zu beschädigen. Vor dem

eigentlichen JSON-Parsing muss dieser Wert daher zunächst dekodiert werden – mit `unbase64()` in einen Binärwert und anschließend mit `CAST` in einen lesbaren String:

```
CREATE OR REPLACE TABLE kafka_events_bronze_decoded AS
SELECT
  cast(unbase64(key) AS STRING) AS decoded_key,
  offset, partition, timestamp, topic,
  cast(unbase64(value) AS STRING) AS decoded_value -- jetzt ein lesbarer JSON-String
FROM kafka_events_bronze_raw;
```

2.4 Ausblick: der VARIANT-Datentyp

Als dritte, neuere Alternative bietet Databricks den offenen **VARIANT**-Datentyp an. Er speichert beliebig strukturierte semi-strukturierte Daten ohne starres Schema, bietet dabei aber ein deutlich besseres Abfrageverhalten als eine reine STRING-Spalte. Mit `parse_json()` wird eine JSON-Spalte in VARIANT umgewandelt, der Zugriff erfolgt anschließend wie gewohnt über die Doppelpunkt-Syntax:

```
CREATE OR REPLACE TABLE kafka_events_bronze_variant AS
SELECT
  decoded_key, offset, partition, timestamp, topic,
  parse_json(decoded_value) AS json_variant_value
FROM kafka_events_bronze_decoded;

SELECT
  json_variant_value:device :: STRING,
  json_variant_value:items
FROM kafka_events_bronze_variant;
```

Ergänzt aus der offiziellen Databricks-Dokumentation: `read_files()` ist die empfohlene, formatunabhängige Tabellenfunktion zum Einlesen von CSV, JSON, XML, TEXT, PARQUET, AVRO, ORC und Binärdateien; die konkreten Optionsnamen für jedes Format (z. B. `sep`, `header`, `encoding`, `quote`, `escape` bei CSV, `multiLine` bei JSON) entsprechen dabei den bekannten `DataFrameReader`-Optionen von Apache Spark. Der VARIANT-Datentyp, der in diesem Kurs noch als Public Preview beschrieben wird, ist inzwischen für die meisten Compute-Typen allgemein verfügbar (GA) und gilt bei neuen JSON-lastigen Ingestion-Pipelines zunehmend als bevorzugte Alternative zu manuell definierten STRUCT-Schemas, insbesondere wenn sich die Struktur der Quelldaten häufig ändert.

Quelle: [read_files table-valued function – Databricks-Dokumentation](#), [Read and write JSON files – Databricks-Dokumentation](#)

